DIFFHLS: Differential Learning for High-Level Synthesis QoR Prediction with GNNs and LLM Code Embeddings

Zedong Peng
Shanghai Jiao Tong University
Shanghai, China
zedongpeng@sjtu.edu.cn

Zeju Li
The Chinese University of Hong Kong
Hong Kong, China
lizeju@link.cuhk.edu.hk

Qiang Xu
The Chinese University of Hong Kong
Hong Kong, China
qxu@cse.cuhk.edu.hk

Jieru Zhao
Shanghai Jiao Tong University
Shanghai, China
zhao-jieru@sjtu.edu.cn

***Abstract*—High-Level Synthesis (HLS) compiles C/C++ into RTL, but exploring pragma-driven optimization choices remains expensive because each design point requires time-consuming synthesis. We propose DIFFHLS, a differential learning framework for HLS Quality-of-Result (QoR) prediction that learns from kernel–design pairs: a kernel baseline and a pragma-inserted design variant. DIFFHLS encodes kernel and design intermediate-representation graphs with dedicated graph neural network (GNN) branches, and augments the delta pathway with code embeddings from a pretrained code large language model (LLM). Instead of regressing absolute targets directly, we jointly predict the kernel baseline and the design-induced delta, and compose them to obtain the design prediction. On PolyBench, DIFFHLS attains lower average MAPE than GNN baselines under four GNN backbones, and LLM code embeddings consistently improve over a GNN-only ablation. We further validate scalability on the ForgeHLS dataset.**

***Index Terms*—high-level synthesis, QoR prediction, graph neural networks, differential learning, large language models**

I. INTRODUCTION

HLS compiles C/C++ into RTL and is widely used to build FPGA accelerators. In practice, designers explore pragma choices such as pipelining and unrolling to trade off performance and resource usage, but each design point can require a costly HLS run.

QoR prediction estimates post-HLS design metrics to accelerate design space exploration (DSE); typical targets span resource utilization metrics such as DSP, FF, and LUT, and timing metrics such as the critical-path delay (CP). GNN-based predictors over IR graphs are effective [1], [2], but they must handle large inter-kernel target variance and may miss pragma-aware semantic cues that remain explicit in source code.

This paper introduces **DIFFHLS**, a differential QoR predictor that learns from kernel–design pairs. We treat the original kernel as a baseline and a pragma-inserted implementation as a design variant, and train the model to predict both the kernel baseline and the design-induced delta. We further incorporate frozen LLM code embeddings to complement IR graph signals. The key design rationale behind DIFFHLS rests on two observations:

1) Predicting changes is more precise than predicting absolutes. In pragma-driven datasets, each kernel induces a cluster of design points that share a large kernel-dependent baseline but differ due to pragma choices. Directly regressing the design-level target forces the model to simultaneously explain the large baseline differences across kernels and the smaller, pragma-driven variations within each kernel. By decomposing the prediction into a kernel baseline and a design-induced delta, we reduce target variance within each cluster, let the model focus on pragma effects, and gain a natural diagnostic: errors can be attributed to either baseline estimation or delta modeling.

2) Graphs and code carry different but complementary information. In HLS optimization, pragma directives and code-level patterns such as loop structure and memory-access style strongly influence resource usage. IR graphs capture control and data dependencies across the program, while source code retains explicit pragma annotations and local patterns that may be weakened during IR lowering. We therefore combine graph representations with pretrained LLM code embeddings: graphs model structural dependencies, and code embeddings provide pragma-aware semantic context.

Our contributions are:

- We propose DIFFHLS, a differential learning formulation for HLS QoR prediction that models each pragma-inserted design as a kernel baseline plus a learned delta.
- We design a hybrid predictor that fuses CDFG-based GNN representations with pretrained LLM code embeddings to capture both IR-level structure and pragma-aware code semantics.
- We evaluate on PolyBench with targeted ablations that quantify the contributions of differential learning, code

embeddings, and component-level error sources, and further validate scalability on the larger ForgeHLS dataset.

II. Related Work

A. HLS QoR Prediction

QoR prediction for HLS has been widely studied. Common benchmarks include PolyBench/C [3], DB4HLS [4], and HLSyn [5]. Larger-scale datasets such as ForgeHLS [6] and HLS4ML [7] expand diversity. Early work leverages hand-crafted features from HLS reports or compiler IR with classical regressors [8]–[10]. Recent approaches represent programs as IR graphs and apply GNNs; representative examples include CDFG-based prediction [1], hierarchical and heterogeneous extensions [2], [11]–[13], post-route estimation [14], and GNN-based area and timing prediction [15]. Hierarchy-aware encoders target long-range dependencies from nested loops [16], [17]; related modeling has been studied for pre-HLS power prediction [18]. Cross-modal approaches fuse source-code tokens with IR graphs via transformers and GNNs [19]. Mixture-of-experts routing has been proposed for specialized prediction [20]. These methods typically regress absolute QoR targets per design, making it hard to capture the subtle effects of individual pragmas accurately. DiffHLS addresses this by decomposing prediction into a kernel baseline plus a pragma-induced delta.

B. LLM Code Embeddings

Pretrained code models provide useful representations for downstream code understanding; encoder-style models such as CodeBERT [21] and GraphCodeBERT [22] learn semantic embeddings that can complement graph-structured IR features. Since many strong code LMs are decoder-only, recent work studies how to extract encoder-style representations from them [23]; other work explores using LM decoding directly for regression [24]. Code embeddings can complement IR graphs by retaining pragma directives and local semantic cues that may be weakened during IR lowering; this motivates lightweight fusion of pretrained code representations with graph encoders [21], [22]. DiffHLS incorporates a lightweight, frozen LLM code embedding as an auxiliary design feature, avoiding expensive LLM fine-tuning while improving sensitivity to pragma-driven changes.

III. Method

A. Problem Formulation

For each kernel k and one of its pragma-inserted designs d, we predict a QoR target $y_d \in \mathbb{R}$, where the target is one of DSP, FF, LUT, or CP. We treat each design as a paired sample (k, d) and decompose the design-level target into a kernel baseline y_k and a design-induced delta:

$$\Delta = y_d - y_k. \tag{1}$$

This formulation reduces inter-kernel scale variance and centers learning on pragma effects. Both y_k and y_d are obtained from post-HLS synthesis reports: y_k represents the QoR metric of the original kernel code without pragma directives, while y_d represents the corresponding metric of the pragma-inserted design code.

Algorithm 1 Training of DiffHLS

Input: paired samples $\{(\mathcal{G}_k, \mathcal{G}_d, y_k, y_d)\}$, code tokens $\mathbf{t}$
for each minibatch **do**
  $\Delta \leftarrow y_d - y_k$
  $\mathbf{h}_k \leftarrow \text{GNNEnc}_k(\mathcal{G}_k)$; $\mathbf{h}_d \leftarrow \text{GNNEnc}_d(\mathcal{G}_d)$
  if Code Emb. **then**
    $\mathbf{h}_c \leftarrow \text{CodeEnc}(\mathbf{t})$
    $\mathbf{h}_\Delta \leftarrow [\mathbf{h}_k; \mathbf{h}_d; \mathbf{h}_c]$
  else
    $\mathbf{h}_\Delta \leftarrow [\mathbf{h}_k; \mathbf{h}_d]$
  end if
  $\hat{y}_k \leftarrow f_k(\mathbf{h}_k)$; $\widehat{\Delta} \leftarrow f_\Delta(\mathbf{h}_\Delta)$
  $\mathcal{L} \leftarrow \ell(\hat{y}_k, y_k) + \ell(\widehat{\Delta}, \Delta) + \ell(\hat{y}_k + \widehat{\Delta}, y_d)$
  Update parameters using $\nabla\mathcal{L}$
end for

B. Graph Encoding

We build IR graphs $\mathcal{G}_k$ and $\mathcal{G}_d$ for the kernel and design, respectively, following the CDFG (control-data flow graph) representation of Wu *et al.* [1]. The graphs are extracted from the HLS toolchain's ADB intermediate representation, which captures both control flow and data dependencies in a unified graph structure.

We encode them with two parameter-disjoint graph encoders for the kernel and design branches, with the same architecture but independent parameters:

$$\mathbf{h}_k = \text{GNNEnc}_k(\mathcal{G}_k), \qquad \mathbf{h}_d = \text{GNNEnc}_d(\mathcal{G}_d). \tag{2}$$

Here $\mathbf{h}_k, \mathbf{h}_d \in \mathbb{R}^D$. As illustrated in Fig. 1, each encoder stacks N GNN layers, where each layer consists of a graph convolution, followed by GraphNorm and a ReLU nonlinearity. A sum pooling readout aggregates node representations into a graph-level embedding.

We follow Wu *et al.* [1] for node and edge attribute construction.

C. LLM Code Embedding

IR graphs capture compiler-level structure but may miss source-level patterns relevant to optimization. We therefore augment each design d with a frozen code encoder $g(\cdot)$ from a pretrained code LLM. Let $\mathbf{t}$ denote the token sequence obtained by concatenating the design source files. We compute a fixed-length representation by extracting the last token's hidden state:

$$\mathbf{z} = g(\mathbf{t})_{\text{last}}. \tag{3}$$

Since the code embedding dimension L may differ from the GNN hidden size D, we use a lightweight adapter $a : \mathbb{R}^L \to \mathbb{R}^D$:

$$\mathbf{h}_c = a(\mathbf{z}). \tag{4}$$

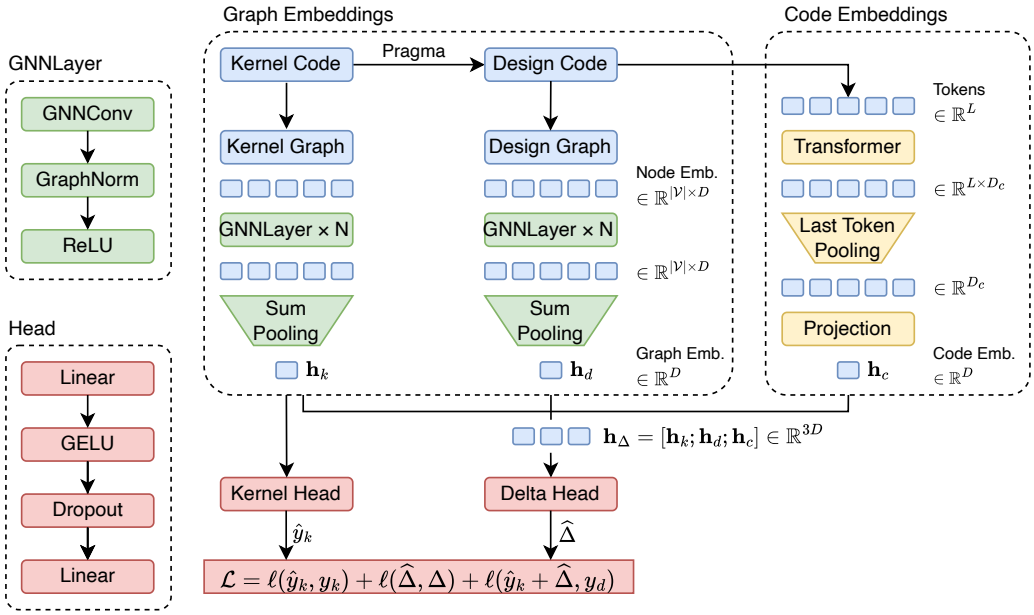


Fig. 1. Overview of DIFFHLS. We encode kernel and design IR graphs, inject LLM code embeddings into the delta pathway, and predict the kernel baseline and delta; finally the design prediction is obtained by composition.

```
void vector_add(int a[8], int b[8], int c[8]) {
   for (int i = 0; i < 8; i++) {
      c[i] = a[i] + b[i];
   }
}
```

Pragma insertion

```
void vector_add(int a[8], int b[8], int c[8]) {
#pragma HLS ARRAY_PARTITION variable=a factor=2
#pragma HLS ARRAY_PARTITION variable=b factor=2
#pragma HLS ARRAY_PARTITION variable=c factor=8
   for (int i = 0; i < 8; i++) {
#pragma HLS PIPELINE OFF
#pragma HLS UNROLL factor=2
      c[i] = a[i] + b[i];
   }
}
```

Fig. 2. Example kernel code (top) and design code (bottom) illustrating HLS pragma insertion.

Thus $\mathbf{h}_c \in \mathbb{R}^D$. We inject LLM code embeddings only into the delta pathway via the delta head, to preserve the interpretation of y_k as a GNN-only kernel baseline.

For each design d, we use its pragma-inserted C/C++ source files as the code input. We concatenate the design files into a single token sequence and feed it to the frozen code encoder. We use only design code because the kernel baseline y_k is predicted from the kernel IR graph alone, while code embeddings serve as an auxiliary signal for the design delta. Figure 2 shows an example kernel and its corresponding pragma-inserted design.

D. Differential Heads

To implement the decomposition in Eq. (1), we use two prediction heads that are trained jointly but serve distinct roles. The detailed architecture is illustrated in Fig. 1. The kernel head estimates the kernel baseline solely from the kernel embedding:

$$\hat{y}_k = f_k(\mathbf{h}_k), \tag{5}$$

where $f_k : \mathbb{R}^D \to \mathbb{R}$ is a small MLP regressor. Concretely, both heads use the same MLP template, differing only in input dimension.

The delta head targets the design-induced change and therefore conditions on both the kernel and design embeddings, optionally augmented with the code embedding $\mathbf{h}_c$, as shown in Algorithm 1:

$$\mathbf{h}_\Delta = \begin{cases} [\mathbf{h}_k; \mathbf{h}_d], & \text{w/o code emb.} \\ [\mathbf{h}_k; \mathbf{h}_d; \mathbf{h}_c], & \text{w/ code emb.} \end{cases} \tag{6}$$

$$\hat{\Delta} = f_\Delta(\mathbf{h}_\Delta), \tag{7}$$

where $f_\Delta : \mathbb{R}^{2D} \to \mathbb{R}$ or $\mathbb{R}^{3D} \to \mathbb{R}$ is another MLP.

We then compose the final design prediction as:

$$\hat{y}_d = \hat{y}_k + \hat{\Delta}. \tag{8}$$

This two-head design separates the kernel-dependent baseline from pragma-driven changes. Code embeddings enter only through the delta pathway, so $\hat{y}_k$ remains a graph-only kernel baseline.

E. Training Objective

We supervise kernel, delta, and design outputs with a SmoothL1 loss $\ell(\cdot, \cdot)$:

$$\mathcal{L} = \ell(\hat{y}_k, y_k) + \ell(\hat{\Delta}, \Delta) + \ell(\hat{y}_k + \hat{\Delta}, y_d), \tag{9}$$

where all three terms are equally weighted. The first two terms directly supervise the kernel baseline prediction and the delta prediction, respectively. The third term enforces consistency

TABLE I
TARGET VALUE RANGES (KERNEL, DELTA, DESIGN).

[min, max]	Kernel	Delta	Design
DSP	[3, 29]	[−7, 75]	[5, 94]
FF	[495, 3994]	[−174, 25414]	[1296, 28967]
LUT	[1214, 5192]	[151, 35865]	[1842, 39464]
CP	[5.31, 7.21]	[−0.10, 0.78]	[5.55, 7.75]

TABLE II
QOR PREDICTION ACCURACY UNDER DIFFERENT GNN BACKBONES.

Backbone	Method	DSP	FF	LUT	CP
PNA	GNN [1]	14.71	26.47	22.86	8.87
	HGNN [2]	7.84	9.65	10.55	-
	DIFFHLS	**3.63**	**8.18**	**5.52**	**1.34**
GraphSAGE	GNN [1]	17.01	39.11	28.09	8.25
	HGNN [2]	6.94	9.99	9.86	-
	DIFFHLS	**3.31**	**7.34**	**5.12**	**1.06**
GCN	GNN [1]	25.30	38.34	28.64	8.79
	HGNN [2]	**7.45**	11.24	11.27	-
	DIFFHLS	8.10	**9.31**	**5.46**	**1.37**
GAT	GNN [1]	28.66	54.73	46.19	10.32
	HGNN [2]	8.18	11.57	12.34	-
	DIFFHLS	**6.41**	**9.63**	**6.11**	**1.53**

We report MAPE in %. Best results are highlighted in **bold**.

between the composed prediction $\hat{y}_k + \widehat{\Delta}$ and the ground-truth design target y_d.

This three-term objective serves two purposes. First, it ensures that both the kernel head and delta head learn meaningful predictions individually. Second, it prevents error accumulation when composing the two predictions. Without the consistency term, errors in $\hat{y}_k$ and $\widehat{\Delta}$ could accumulate and produce larger errors in the final design prediction. The consistency term $\ell(\hat{y}_k + \widehat{\Delta}, y_d)$ penalizes such misalignment and encourages the two heads to cooperate.

IV. EXPERIMENTAL SETUP

A. Metrics

We report mean absolute error, denoted MAE; mean absolute percentage error, denoted MAPE and reported in percent; and coefficient of determination, denoted R^2. Lower is better for MAE and MAPE, and higher is better for R^2. For a set of N designs with targets $\{y_i\}$ and predictions $\{\hat{y}_i\}$, MAE, MAPE, and R^2 are:

$$\begin{aligned} \mathrm{MAE} &= \frac{1}{N}\sum_{i=1}^{N} |y_i - \hat{y}_i|, \\ \mathrm{MAPE} &= \frac{100}{N_0}\sum_{i:\, y_i \neq 0} \left|\frac{y_i - \hat{y}_i}{y_i}\right|, \\ R^2 &= 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2}. \end{aligned} \tag{10}$$

Here $N_0 = \big|\{i : y_i \neq 0\}\big|$, $\bar{y} = \frac{1}{N}\sum_{i=1}^{N} y_i$, and MAPE excludes samples with zero targets to avoid division-by-zero.

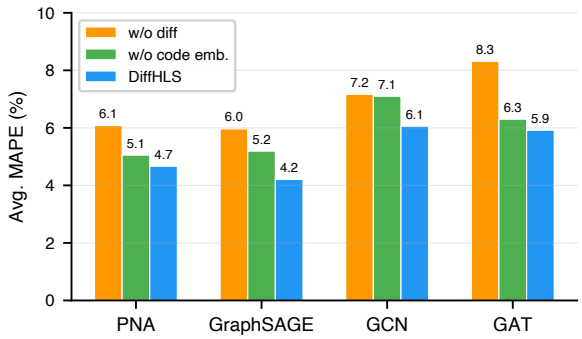


Fig. 3. Average MAPE (%) across DSP, FF, LUT, and CP for the three method variants under four GNN backbones.

B. Dataset

We evaluate on 27 PolyBench/C kernels [3] and 10,108 design points derived from the ForgeHLS PolyBench subset [6] via pragma insertion. We predict design-level QoR targets $y_d \in \{\mathrm{DSP}, \mathrm{FF}, \mathrm{LUT}, \mathrm{CP}\}$, whose value ranges are listed in Table I. We use a design-level random split with an 8:1:1 train/validation/test ratio over paired kernel–design samples. We train separate models for each of DSP, FF, LUT, and CP.

C. Implementation Details

Unless otherwise stated, we train with Adam optimizer, learning rate 5×10^{-4}, batch size 16, hidden size 128, two GNN layers, and dropout 0.02. We use ReduceLROnPlateau with patience 15 and factor 0.8, and select the best checkpoint based on validation loss. All experiments are run on a single NVIDIA RTX 4090 GPU with 24 GB memory.

V. EXPERIMENTAL RESULTS

A. Main Results

Table II summarizes design-level QoR prediction MAPE under four GNN backbones. DIFFHLS achieves strong performance across all backbones, with GraphSAGE yielding the best results. Compared to the GNN baseline [1], DIFFHLS reduces MAPE substantially on every target under all backbones. For example, under GraphSAGE, DSP MAPE drops from 17.01% to 3.31% and LUT MAPE drops from 28.09% to 5.12%.

B. Differential vs. Direct Prediction

Table III compares DIFFHLS with a direct prediction variant (DIFFHLS w/o diff) that disables the differential formulation and directly predicts design targets. Disabling differential learning consistently increases MAPE on FF and LUT across all backbones. The degradation is particularly large on LUT under GAT, where MAPE increases from 6.11% to 13.39%.

This validates our core hypothesis: by decomposing $y_d = y_k + \Delta$, the model only needs to learn the pragma-induced change Δ rather than the full target value, which varies by orders of magnitude across different kernels. The differential formulation therefore simplifies the learning problem considerably. Fig. 3 visualizes the average MAPE of the three variants across all backbones.

TABLE III
ABLATION STUDY: DIFFERENTIAL VS. DIRECT PREDICTION AND EFFECT OF CODE EMBEDDINGS.

Backbone	Method	DSP			FF			LUT			CP		
		MAPE ↓	MAE ↓	R^2 ↑	MAPE ↓	MAE ↓	R^2 ↑	MAPE ↓	MAE ↓	R^2 ↑	MAPE ↓	MAE ↓	R^2 ↑
PNA	DIFFHLS (w/o diff)	3.86	**1.16**	**0.980**	10.75	725.94	0.972	8.50	891.66	0.969	**1.22**	**0.090**	0.711
	DIFFHLS (w/o code emb.)	4.18	1.39	0.976	8.51	677.34	0.963	6.30	641.35	0.967	**1.22**	**0.090**	**0.735**
	DIFFHLS	**3.63**	1.21	0.899	**8.18**	**629.84**	**0.973**	**5.52**	**627.03**	**0.989**	1.34	0.099	0.710
GraphSAGE	DIFFHLS (w/o diff)	4.19	**1.25**	**0.971**	10.60	680.01	0.926	7.76	707.05	0.935	1.31	0.096	0.679
	DIFFHLS (w/o code emb.)	5.11	1.81	0.945	8.64	698.86	**0.974**	5.78	576.78	**0.991**	1.24	0.092	0.721
	DIFFHLS	**3.31**	1.28	0.964	**7.34**	**647.85**	0.897	**5.12**	**510.83**	**0.991**	**1.06**	**0.078**	**0.847**
GCN	DIFFHLS (w/o diff)	8.91	**2.78**	0.937	10.71	**897.59**	**0.935**	7.83	776.59	**0.986**	**1.22**	**0.090**	0.718
	DIFFHLS (w/o code emb.)	10.08	3.34	0.915	10.37	996.00	0.678	6.68	751.33	0.981	1.27	0.094	**0.732**
	DIFFHLS	**8.10**	2.84	**0.950**	**9.31**	1001.49	0.413	**5.46**	**621.47**	0.985	1.37	0.101	0.664
GAT	DIFFHLS (w/o diff)	7.67	2.57	0.938	10.88	820.99	0.947	13.39	1186.50	0.969	1.32	0.098	0.294
	DIFFHLS (w/o code emb.)	8.07	2.60	0.952	9.98	**763.87**	**0.967**	**5.88**	**562.37**	**0.991**	**1.28**	**0.095**	**0.700**
	DIFFHLS	**6.41**	**2.18**	**0.958**	**9.63**	905.99	0.863	6.11	715.22	0.982	1.53	0.113	0.628

DIFFHLS (w/o diff) disables differential learning and directly predicts design targets without the kernel–design delta decomposition.
DIFFHLS (w/o code emb.) disables code embeddings.

TABLE IV
ABLATION STUDY: PREDICTION HEAD ACCURACY.

Head	Backbone	Code Emb.	DSP			FF			LUT			CP		
			MAPE	MAE	R^2	MAPE	MAE	R^2	MAPE	MAE	R^2	MAPE	MAE	R^2
Kernel $\hat{y}_k$	PNA	No	**0.43**	**0.08**	**1.000**	0.74	15.53	0.999	0.55	13.36	1.000	**0.06**	**0.004**	**0.993**
		Yes	0.84	0.12	0.999	**0.66**	**15.08**	**1.000**	**0.38**	**8.81**	**1.000**	0.12	0.008	0.933
	GraphSAGE	No	0.89	0.14	0.999	0.94	**20.13**	1.000	**0.58**	**13.54**	1.000	0.11	0.008	0.955
		Yes	**0.51**	**0.09**	**1.000**	**0.86**	20.48	1.000	0.61	13.83	1.000	**0.07**	**0.005**	**0.998**
	GCN	No	1.20	0.15	**0.999**	**0.67**	**13.44**	1.000	0.82	19.48	0.999	**0.06**	**0.004**	**0.996**
		Yes	**0.86**	**0.14**	0.999	0.74	16.36	**1.000**	**0.38**	**9.05**	**1.000**	0.24	0.017	0.379
	GAT	No	**0.79**	**0.12**	**0.999**	2.09	51.36	0.994	**0.91**	**22.56**	**0.999**	**0.10**	**0.007**	**0.992**
		Yes	1.63	0.25	0.997	**1.01**	**21.56**	**0.999**	1.16	30.21	0.998	0.12	0.008	0.971
Delta $\widehat{\Delta}$	PNA	No	13.55	1.38	**0.971**	**58.52**	676.20	0.961	13.71	641.63	0.965	**35.80**	**0.090**	**0.709**
		Yes	**11.37**	**1.23**	0.872	66.76	**631.17**	**0.972**	**12.07**	**627.14**	**0.988**	38.13	0.099	0.654
	GraphSAGE	No	18.59	1.81	0.931	53.23	695.03	**0.973**	11.83	576.72	0.990	36.61	0.092	0.706
		Yes	**11.92**	**1.28**	**0.956**	**46.15**	**650.10**	0.891	**10.63**	**511.54**	**0.991**	**31.33**	**0.078**	**0.740**
	GCN	No	38.82	3.37	0.895	59.39	**995.09**	**0.661**	13.60	754.88	0.980	**36.13**	**0.093**	**0.675**
		Yes	**26.43**	**2.86**	**0.938**	**32.41**	1002.12	0.384	**11.72**	**622.58**	**0.984**	37.80	0.102	0.638
	GAT	No	29.27	2.59	0.939	125.26	**754.27**	**0.966**	**13.14**	**563.01**	**0.991**	**33.77**	**0.094**	**0.657**
		Yes	**22.45**	**2.17**	**0.948**	**59.00**	906.03	0.857	12.63	713.02	0.981	39.63	0.112	0.585

C. Effect of Code Embeddings

Table III also compares DIFFHLS with a GNN-only variant (DIFFHLS w/o code emb.) that removes code embeddings from the delta pathway. Adding code embeddings consistently decreases MAPE across DSP, FF, and LUT under all backbones. The largest impact is on DSP, where MAPE decreases from 5.11% to 3.31% under GraphSAGE.

This suggests that LLM code embeddings capture pragma annotations and loop structures that may be lost during IR generation, providing complementary cues to the graph-based features.

D. Prediction Head Accuracy

Table IV reports accuracy for the kernel head and delta head separately. The kernel head achieves MAPE below 2% and R^2 above 0.99 on DSP, FF, and LUT, confirming that kernel baselines are well-captured by IR graph structure alone. The delta head is more challenging due to the wide range of pragma-induced QoR changes, but still maintains reasonable R^2. Adding code embeddings reduces delta MAPE on DSP from 13.55% to 11.37% under PNA and from 18.59% to 11.92% under GraphSAGE. These results confirm that the kernel baseline is an easy subproblem, while the delta head handles the harder pragma-driven variation.

E. GNN Backbone

Across all experiments, GraphSAGE consistently yields the best results, outperforming PNA, GCN, and GAT on all QoR targets. The differential formulation benefits all four backbones substantially, and code embeddings provide further gains under GraphSAGE. Overall, DIFFHLS is robust to the choice of GNN backbone.

F. Code LLM Backbone

Table V compares two code LLM backbones for embedding extraction under the same training protocol. Under PNA and GraphSAGE, Qwen2.5-Coder-1.5B achieves lower MAPE on DSP and LUT, while Llama-3.2-1B sometimes yields better MAE and R^2 on FF. Under GCN, Llama-3.2-1B achieves lower MAPE on DSP, FF, and CP, while Qwen2.5-Coder-1.5B retains an advantage on LUT. Under GAT, Llama-3.2-1B outperforms

TABLE V
ABLATION STUDY: CODE LLM BACKBONE.

Backbone	Model	DSP			FF			LUT			CP		
		MAPE ↓	MAE ↓	R^2 ↑	MAPE ↓	MAE ↓	R^2 ↑	MAPE ↓	MAE ↓	R^2 ↑	MAPE ↓	MAE ↓	R^2 ↑
PNA	Qwen2.5-Coder-1.5B	**3.63**	**1.21**	0.899	8.18	629.84	**0.973**	**5.52**	627.03	**0.989**	1.34	0.099	0.710
	Llama-3.2-1B	3.96	1.40	**0.979**	**8.09**	**524.57**	0.967	5.60	**457.07**	0.984	**1.13**	**0.084**	**0.776**
GraphSAGE	Qwen2.5-Coder-1.5B	**3.31**	**1.28**	**0.964**	**7.34**	647.85	0.897	**5.12**	510.83	**0.991**	**1.06**	**0.078**	**0.847**
	Llama-3.2-1B	4.52	1.71	0.926	8.14	**528.27**	**0.963**	5.56	**453.39**	0.989	1.17	0.086	0.738
GCN	Qwen2.5-Coder-1.5B	8.10	2.84	**0.950**	9.31	1001.49	0.413	**5.46**	**621.47**	0.985	1.37	0.101	0.664
	Llama-3.2-1B	**8.08**	**2.73**	0.947	**8.93**	**817.44**	**0.947**	6.50	717.54	**0.988**	**1.13**	**0.083**	**0.774**
GAT	Qwen2.5-Coder-1.5B	6.41	2.18	**0.958**	9.63	905.99	0.863	6.11	715.22	0.982	1.53	0.113	**0.628**
	Llama-3.2-1B	**5.91**	**2.09**	0.957	**9.31**	**685.07**	**0.958**	**6.06**	**602.63**	**0.991**	**1.20**	**0.088**	0.628

TABLE VI
FORGEHLS LARGE-SCALE EVALUATION.

Backbone	DSP	FF	LUT	CP
PNA	5.70	25.75	10.05	**4.21**
GraphSAGE	**5.69**	**17.30**	**7.77**	4.93
GCN	9.11	25.18	11.94	5.83
GAT	10.75	20.16	9.27	4.41

We report design-level MAPE in %.

Qwen2.5-Coder-1.5B on most metrics. These results suggest that the optimal code backbone may depend on the GNN architecture and target metric. Overall, we use Qwen2.5-Coder-1.5B as the default code embedding backbone.

G. ForgeHLS Large-Scale Evaluation

To further validate scalability, we evaluate DIFFHLS on the ForgeHLS large-scale dataset [6]. All runs use the full DIFFHLS configuration with differential heads and code embeddings enabled.

Table VI reports design-level MAPE for four GNN backbones. GraphSAGE delivers the strongest overall performance, achieving the lowest MAPE on resource targets. PNA achieves the lowest CP MAPE. The ranking among backbones is consistent with the PolyBench results, confirming that DIFFHLS generalizes to a substantially larger and more diverse design space.

VI. CONCLUSION

We presented DIFFHLS, a differential learning framework for HLS QoR prediction that learns from kernel–design pairs by decomposing design-level targets into a kernel baseline and a design-induced delta, and augments graph representations with LLM code embeddings. On PolyBench, DIFFHLS improves design-level prediction accuracy over baselines under four GNN backbones across all four QoR targets. Our ablations show that differential learning provides the largest gain, and code embeddings further improve performance. We also validate scalability on the larger ForgeHLS dataset.

REFERENCES

[1] N. Wu, H. Yang, Y. Xie, P. Li, and C. Hao, "High-level synthesis performance prediction using graph neural networks: Benchmarking, modeling, and advancing," in *Proceedings of the ACM/IEEE Design Automation Conference (DAC)*, 2022.

[2] M. Gao, J. Zhao, Z. Lin, and M. Guo, "Hierarchical source-to-post-route qor prediction in high-level synthesis with gnns," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2024, pp. 1–6.

[3] L.-N. Pouchet and T. Yuki, "Polybench/c 4.2," https://sourceforge.net/projects/polybench/files/polybench-c-4.2/, 2016, accessed: 2025-11-20.

[4] L. Ferretti, J. Kwon, G. Ansaloni, G. Di Guglielmo, L. Carloni, and L. Pozzi, "Db4hls: A database of high-level synthesis design space explorations," *IEEE Embedded Systems Letters*, vol. 13, no. 4, pp. 194–197, 2021.

[5] Y. Bai, A. Sohrabizadeh, Z. Qin, Z. Hu, Y. Sun, and J. Cong, "Towards a comprehensive benchmark for high-level synthesis targeted to FPGAs," *Advances in Neural Information Processing Systems*, vol. 36, pp. 45 288–45 299, 2023.

[6] Z. Peng, Z. Li, M. Gao, Q. Xu, C. Zhang, and J. Zhao, "ForgeHLS: A Large-Scale, Open-Source Dataset for High-Level Synthesis," Aug. 2025, arXiv:2507.03255 [cs]. [Online]. Available: http://arxiv.org/abs/2507.03255

[7] B. Hawks, J. Weitz, D. Demler, K. Tame-Narvaez, D. Plotnikov, M. M. Rahimifar, H. E. Rahali, A. C. Therrien, D. Sproule, E. E. Khoda *et al.*, "wa-hls4ml: A benchmark and surrogate models for hls4ml resource and latency estimation," *arXiv preprint arXiv:2511.05615*, 2025.

[8] G. Zhong, A. Prakash, S. Wang, Y. Liang, T. Mitra, and S. Niar, "Design space exploration of fpga-based accelerators with multi-level parallelism," in *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*. IEEE, 2017, pp. 1141–1146.

[9] S. Dai, Y. Zhou, H. Zhang, E. Ustun, E. F. Young, and Z. Zhang, "Fast and accurate estimation of quality of results in high-level synthesis with machine learning," in *2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2018, pp. 129–132.

[10] J. Zhao, L. Feng, S. Sinha, W. Zhang, Y. Liang, and B. He, "Comba: A comprehensive model-based analysis framework for high level synthesis of real applications," in *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2017, pp. 430–437.

[11] A. Sohrabizadeh, Y. Bai, Y. Sun, and J. Cong, "Automated accelerator optimization aided by graph neural networks," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 55–60.

[12] Z. Lin, Z. Yuan, J. Zhao, W. Zhang, H. Wang, and Y. Tian, "Powergear: Early-stage power estimation in fpga hls via heterogeneous edge-centric gnns," in *Procs. of Design, Automation and Test in Europe Conference and Exhibition (DATE)*, 2022.

[13] Z. Lin, J. Zhao, S. Sinha, and W. Zhang, "Hl-pow: A learning-based power modeling framework for high-level synthesis," in *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2020, pp. 574–580.

[14] P. Goswami and D. Bhatia, "Predicting post-route quality of results estimates for HLS designs using machine learning," in *2022 23rd International Symposium on Quality Electronic Design (ISQED)*. IEEE, 2022, pp. 45–50.

[15] M. U. Jamal, Z. Li, M. T. Lazarescu, and L. Lavagno, "A graph neural network model for fast and accurate quality of result estimation for high-level synthesis," *IEEE Access*, 2023.

[16] A. Sohrabizadeh, Y. Bai, Y. Sun, and J. Cong, "Robust GNN-based representation learning for HLS," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–9.

[17] E. Murphy and L. Josipović, "Balor: Hls source code evaluator based on custom graphs and hierarchical gnns," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024, pp. 1–9.

[18] Z. Lin, Z. Peng, M. Gao, J. Zhao, and Z. Lin, "Hippo: A hierarchy-preserving and noise-tolerant pre-hls power modeling framework for fpga," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2025.

[19] Z. Qin, Y. Bai, A. Sohrabizadeh, Z. Ding, Z. Hu, Y. Sun, and J. Cong, "Cross-modality program representation learning for electronic design automation with high-level synthesis," *arXiv preprint arXiv:2406.09606*, 2024.

[20] W. Li, D. Wang, Z. Ding, A. Sohrabizadeh, Z. Qin, J. Cong, and Y. Sun, "Hierarchical mixture of experts: Generalizable learning for high-level synthesis," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 17, 2025, pp. 18 476–18 484.

[21] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, and D. Jiang, "Codebert: A pre-trained model for programming and natural languages," *arXiv preprint arXiv:2002.08155*, 2020.

[22] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu *et al.*, "Graphcodebert: Pre-training code representations with data flow," *arXiv preprint arXiv:2009.08366*, 2020.

[23] P. Suganthan, F. Moiseev, L. Yan *et al.*, "Adapting decoder-based language models for diverse encoder downstream tasks," 2025, arXiv preprint.

[24] X. Song and D. Bahri, "Decoding-based regression," 2025, arXiv preprint.